PRECISION

BY

DOUGLAS QUIGG

RESEARCH AND THEORY
CERTIFICATES INSTEAD OF TABLES, AND THE CONSTANTS NOBODY CHECKS

ORIOR
2026

© 2026 Douglas Quigg
AGPL-3.0-or-later OR LicenseRef-Commercial OR LicenseRef-Educational

ABSTRACT

A number theoretic transform is correct only if its modulus is prime and its twiddle root has order exactly the transform length. Both facts are proved here at the point of use, by one Proth witness for primality and two exponentiations for the order, on unbounded integers at any width. A 1,233-digit modulus is proved prime in under three tenths of a second, with the witness recorded beside the verdict. A root of half the required order passes every invariant computable from the twiddle table in $O(n)$, and the order test is the only one that separates it. Two silent wrong answers fall to the same discipline: a composite modulus of the correct Proth shape, and a 32-bit overflow in a device butterfly.

An integer relation search is measured as a function of its precision. Its cost grows with an exponent rising to 2.64 per doubling, close to the 2.585 a Karatsuba multiply on operands linear in the precision predicts, while the lattice reduction grows no faster than the multiply count. The precision at which a device multiply overtakes the host is one at which a single search cell takes about a day, and no faster device closes that gap.

The record of configurations states each claim with the result that would refute it, written before the verdict, and the constraints every entry is bounded by.

CONTENTS

1 THE TWIDDLE CONSTANTS, PROVED AND NOT COPIED 1
1.1 What a twiddle constant is 1
1.2 The attack, and why it works 1
1.3 The same fault, with no attacker 1
1.4 Proth's theorem makes the check a proof 2
1.5 The order of the root, exactly, without factoring anything 3
1.6 What it costs on one desktop 3
1.7 Why the size is ours to choose 3
1.8 The group law, and the proof that it is not enough on its own 4
1.8.1 The disproof: the law is necessary and is not sufficient 4
1.8.2 Why the order of the checks matters operationally 5
1.9 On the device, a second silent wrong answer 5
1.10 What it costs on the device 6
1.11 What this refuses 6
1.12 Availability 6

2 THE RELATION SEARCH, AND WHERE THE DEVICE STOPS HELPING 7
2.1 What the search returns, and why that matters 7
2.2 The measurement 7
2.3 Which part carries the cost 8
2.4 The consequence 8
2.5 What this does not settle 9

3 THE RECORD OF CONFIGURATIONS 11
3.1 The schema 11
3.2 Located constraints 11

BIBLIOGRAPHY 13

1

THE TWIDDLE CONSTANTS, PROVED AND NOT COPIED

This chapter states what it costs to prove a transform's constants instead of inheriting them, what that proof catches, and why every answer it gives carries the witness establishing it. The same result, placed against the verification literature, is the twiddle constants research paper [6].

A number theoretic transform is a sum of powers of one constant. Everything the transform does depends on that constant having exactly the order it is supposed to have, and almost nobody checks.

1.1 WHAT A TWIDDLE CONSTANT IS

The transform of a sequence of length n over the integers modulo q is

$$\hat{p}_j = \sum_i p_i \, \omega^{ij}$$

where ω is an n-th root of unity modulo q. The powers of ω the butterflies evaluate are the **twiddle constants**. In the Cooley-Tukey butterfly they enter as $c = a + bw$ and $d = a - bw$; in the Gentleman-Sande butterfly as $c = a + b$ and $d = (a - b)w$. A transform of length 2^k is k stages of $n/2$ butterflies. The twiddles are touched more often than any other value in the computation and are the only inputs supposed to be constant.

1.2 THE ATTACK, AND WHY IT WORKS

Ravi, Yang, Bhasin, Zhang and Chattopadhyay [7] inject a single electromagnetic fault setting the twiddle constants to zero. The transform does not fail. It returns a value of the ordinary shape carrying far less entropy than it should, and that is enough to recover a Kyber key and to forge a Dilithium signature, including a bypass of Dilithium's own verification.

The attack works because **nothing downstream can tell a real root of unity from a fiddled one**. The output of a transform with broken twiddles is the same size, the same type and the same range as the output of a correct one. There is no error to raise. A wrong twiddle is not detected, it is used.

1.3 THE SAME FAULT, WITH NO ATTACKER

The fault condition also arises with no attacker, and more cheaply.

A transform is built on three moduli. The third, 1610612737, is taken on faith because it has the right shape, $3 \cdot 2^{29} + 1$, the form such a modulus must have. Its generator is raised to $(p-1)/L$ to produce a root of unity of order L, exactly as the construction requires, and a check written beside it reports the root is fine.

The check is trial division to one hundred, on a ten digit number. $1610612737 = 79 \cdot 20387503$ is **composite**, and the proof is one line: $2^{1610612736} \not\equiv 1 \pmod{1610612737}$. Fermat refuses it. The certificate is the witness 2.

Because it is composite, $g^{(p-1)/L}$ has no particular order, the twiddles are wrong, and the transform returns numbers of the ordinary shape that are not the convolution of anything. The two correct moduli in the same set return zero wrong coefficients out of thirty-two. The composite one returns thirty-two out of thirty-two, and the failure shows only because the three are recombined against each other and disagree.

A copied constant is a constant nobody checked. The moduli general purpose libraries reach for are the same values everywhere, carried between codebases as literals. The tables are correct, and that is not the point. The point is that correctness is being inherited instead of established, and inherited correctness cannot tell you when a value has been fiddled.

1.4 PROTH'S THEOREM MAKES THE CHECK A PROOF

Theorem 1.1 (Proth [5]). *Let $N = k \cdot 2^n + 1$ with k odd and $k < 2^n$. Then N is prime if and only if there is an integer a with $a^{(N-1)/2} \equiv -1 \pmod{N}$.*

Every modulus such a transform can use has this shape, because admitting a root of unity of order 2^n is the same as 2^n dividing $N - 1$. The theorem therefore reaches every candidate that matters.

Why a witness is a proof. Suppose $a^{(N-1)/2} \equiv -1 \pmod{N}$, and let p be any prime factor of N. Then $a^{(N-1)/2} \equiv -1 \pmod{p}$ as well. The order of a modulo p divides $N - 1$ and does not divide $(N-1)/2 = k \cdot 2^{n-1}$. Since $N - 1 = k \cdot 2^n$, the order must carry the full 2^n, giving $2^n \mid p - 1$ and so $p \equiv 1 \pmod{2^n}$. Every prime factor of N is at least $2^n + 1$.

If N were composite it would have at least two such factors. $N \geq (2^n + 1)^2 > 2^{2n}$. But $k < 2^n$ gives $N = k \cdot 2^n + 1 < 2^{2n} + 1$. The two cannot both hold. N is prime. □

That last step needs the condition $k < 2^n$, and a modulus outside that range is reported as out of range instead of being tested anyway.

Why a witness is easy to find. If N is prime then Euler's criterion makes $a^{(N-1)/2}$ the Legendre symbol of a, which is -1 for every quadratic non-residue, and that is half of all a. A witness is therefore found by trying small integers, and the first one usually works.

Both verdicts are certificates. A witness a with $a^{(N-1)/2} \equiv -1$ proves primality. A witness a with $a^{N-1} \not\equiv 1$ proves compositeness, by Fermat. Neither answer is a probability and neither is a judgement call. The only other outcome is running out of candidates, reported as inconclusive instead of rounded toward either verdict.

1.5 THE ORDER OF THE ROOT, EXACTLY, WITHOUT FACTORING ANYTHING

Proving the modulus prime is not enough. The twiddle is $\omega = g^{(p-1)/L}$, and the step from a generator to a root of unity is where the constant stops being checked.

Proposition 1.2. *For $L = 2^m$, if $\omega^L = 1$ and $\omega^{L/2} \neq 1$, then ω has order exactly L.*

Proof. The order divides $L = 2^m$. It is 2^j for some $j \leq m$. If $j < m$ then 2^j divides $2^{m-1} = L/2$, which would give $\omega^{L/2} = 1$. It does not. $j = m$. □

Two exponentiations settle it completely, with no factoring of anything. This is cheap enough to run on every twiddle before every transform, and it is the step the fault attack removes. It refuses a zeroed twiddle, a root of short order, and a root drawn from a composite modulus, all by the same test.

The generator is proved the same way and just as completely: g generates the whole group if and only if $g^{(p-1)/q} \neq 1$ for every prime q dividing $p-1$. Here $p-1 = k \cdot 2^n$ with k small. Factoring it is factoring k, and the proof is complete instead of partial.

1.6 WHAT IT COSTS ON ONE DESKTOP

One proof is one modular exponentiation, measured on unbounded integers with nothing vectorized and nothing on the device:

bits	decimal digits	one proof
64	19	0.0001 s
128	39	0.0001 s
256	77	0.0002 s
512	154	0.0011 s
1024	308	0.0185 s
2048	617	0.0419 s
4096	1,233	0.2876 s

A **1,233 digit number proved prime in under three tenths of a second**, deterministically. Not a probable prime, not a result with a confidence attached. A certificate, with the witness recorded beside it, letting a reader recheck the claim without rerunning anything.

The cost grows as roughly the square of the bit length, since the exponent has N bits and each squaring costs the multiply. This is the same theorem the large prime searches run, where the records are Proth primes of several million digits, and the difference between those runs and this table is only how long one is willing to wait.

1.7 WHY THE SIZE IS OURS TO CHOOSE

The tabled moduli are small because they are sized to a machine word. A lane holds sixty-four bits, a product of two residues has to fit in one, and the modulus therefore stops below 2^{32}. That is a property of the lane and not of the mathematics.

The arithmetic here is on unbounded integers. The proof runs at any width, and moduli far past anything tabled can be produced and proved on demand:

modulus	shape	transform length
18446744069414584321	$(2^{32}-1)\cdot 2^{32}+1$	2^{32}
9223372195768565761	$2147483685\cdot 2^{32}+1$	2^{32}
396140812571487758203979038 73	$140737488355387\cdot 2^{48}+1$	2^{48}

The last admits a transform of length 281,474,976,710,656. Nobody tables that, because it does not fit in a lane. **The floor belongs to the format, and the format is a choice.**

1.8 THE GROUP LAW, AND THE PROOF THAT IT IS NOT ENOUGH ON ITS OWN

The twiddle table is not a list of numbers that happen to be useful. It is a group, and saying so precisely licenses the way the transform is implemented.

Proposition 1.3 (The law)**.** *Let ω have order exactly n modulo a prime p, and let $w_j = \omega^j$ for $0 \le j < n$. Then $\{w_j\}$ is the cyclic group of order n generated by ω, and $w_j w_k = w_{(j+k) \bmod n}$ for every j and k.*

Proof. $w_j w_k = \omega^{j+k}$. Write $j + k = qn + r$ with $0 \le r < n$. Then $\omega^{j+k} = (\omega^n)^q \omega^r = \omega^r = w_r$, and r is $(j + k) \bmod n$. The identity is $w_0 = 1$ and the inverse of w_j is w_{n-j}. The set is closed, has an identity and has inverses, and so it is a group; it is generated by one element. It is cyclic. □

Two consequences are used directly.

The table sums to zero. For $n > 1$, $\sum_j w_j = (\omega^n - 1)/(\omega - 1) = 0$, exactly, because $\omega \neq 1$ makes the denominator invertible. This is an integer zero and not a small residual.

The table is generated by repeated multiplication. $w_{j+1} = w_j \omega$. Building the whole table costs one modular multiply per entry. In floating point that construction is warned against because error accumulates along the chain. In the exact ring there is no error to accumulate. The cheap construction and the correct one are the same construction.

1.8.1 *The disproof: the law is necessary and is not sufficient*

Proposition 1.4. *Let ω' have order d where d divides n and $d < n$. Then the table $w'_j = \omega'^j$ satisfies the group law at length n exactly, while being the wrong table.*

Proof. Since d divides n, $\omega'^n = (\omega'^d)^{n/d} = 1$. The argument above then runs unchanged with ω' in place of ω, and $w'_j w'_k = w'_{(j+k) \bmod n}$ holds for every j and k. □

A root of half the required order therefore produces a table that is a genuine homomorphic image of the wrong group. At length 4096 with ω' of order 2048:

invariant	verdict on a root of half the order
$w_0 = 1$	passes
$\sum w_j = 0$	passes
$\sum w_j^2 = 0$	passes
group law on sampled pairs	passes, 0 of 256 failures
order test	**fails, the only one that does**

Every invariant computable from the table at $O(n)$ cost is blind to it, because each one is a statement the smaller group also satisfies. What separates them is the order, and the order is not a property any single relation among entries exposes. The separation costs two exponentiations, and it catches the case every cheaper test misses.

1.8.2 *Why the order of the checks matters operationally*

The transform builds its twiddles once by repeated multiplication and reads them at a stride, and the group law permits that. But the group law permits the construction from *any* root whose order divides the length, including a wrong one, and the result would be a transform returning values of the ordinary shape that are not the convolution of anything.

The order proof is therefore not an optional extra beside the optimization. It is the precondition for the optimization being safe, and it has to run first. Stated as a rule: **prove the order, then generate the table.** Reversing those two produces a fast wrong answer.

The change is measured. Replacing an exponentiation per butterfly with one table read cuts the transform at length 2^{27} from 2006 ms to 680 ms, a factor of 2.95, and the output digest is identical at every length tested. A correct optimization moves the clock and not the answer, and that check says which one moved.

1.9 ON THE DEVICE, A SECOND SILENT WRONG ANSWER

The device transform runs over three moduli, each carrying its Proth witness beside it in the source:

modulus	shape	witness	order
2013265921	$15 \cdot 2^{27} + 1$	11	2^{27}
2281701377	$17 \cdot 2^{27} + 1$	3	2^{27}
3892314113	$29 \cdot 2^{27} + 1$	3	2^{27}

Run with 32-bit residues, that kernel agrees with the host on the first modulus and differs in *every coefficient* on the other two. The property separating them is not primality, which all three have. It is that 2013265921 is below 2^{31} and the other two are above it. In the butterfly, two residues below a modulus above 2^{31} sum past 2^{32} and wrap, silently, in a type that raises nothing. The same is true of the borrow in the difference. Only the smallest modulus never reaches the boundary. One arm agrees and two do not.

This is the same fault as the composite modulus, one level down. In both cases an assumption about the arithmetic goes unstated, the transform returns values of the ordinary shape,

and nothing in the output says otherwise. The first is a constant nobody proves; the second is a width nobody checks. Neither is found by reading the code. Both are found because the gate runs cases differing in one property, and the failure names its own cause.

Widened to 64 bits, all three moduli agree with the host, and the device multiply agrees with the reference at every size tested.

1.10 WHAT IT COSTS ON THE DEVICE

One full multiply is two forward transforms, a pointwise product and one inverse, per modulus:

transform length	decimal digits carried	one modulus
2^{20}	10.1 million	8.5 ms
2^{22}	40.4 million	49.4 ms
2^{24}	162 million	220 ms
2^{26}	646 million	938 ms
2^{27}	1.29 billion	2006 ms

Three moduli at 2^{27} is about six seconds of device time for a product of a billion decimal digits. The host's own multiply is Karatsuba, measured at 0.930 s for a million digits and 5.610 s for three million, an exponent of 1.635; extrapolated across three orders of magnitude that is some twenty hours for the same product. The extrapolation is stated as an extrapolation, since it reaches well past anything measured directly.

The packing costs nothing, and that part is worth retention. A large integer is already a run of 32-bit limbs. Handing one to the device and reading one back are the same bytes named twice, and no conversion happens. The device returns the product as three arrays and the caller assembles them at a 32-bit stride. Convolution coefficients run past 2^{32} and have to be carried, and the carrying happens inside that addition, in the host's own big-integer arithmetic. A carry pass written out over a hundred million coefficients would cost more than every transform put together; here there is no pass at all.

1.11 WHAT THIS REFUSES

A modulus outside Proth's range, where k is not below 2^n, since a witness rests on the converse, which does not hold there. A root whose order falls short of the length asked for, the fiddled twiddle exactly. And any verdict without its certificate: every answer carries the witness establishing it.

1.12 AVAILABILITY

The certificate, both rejections and the half-order case are reproduced by an artifact in the orior repository, `https://github.com/dstroy0/orior`.

2

THE RELATION SEARCH, AND WHERE THE DEVICE STOPS HELPING

This chapter measures what an integer relation search costs as a function of the precision it runs at, identifies which part carries that cost, and states the consequence for whether a device can help. Everything below is measured in arithmetic: places of precision, multiplies, operand width, seconds. No figure here counts an operation belonging to any particular subject the search might be pointed at.

2.1 WHAT THE SEARCH RETURNS, AND WHY THAT MATTERS

Almost every measurement in this body of work returns a bound or a null. An integer relation search returns a statement: a closed form either holds or it does not, and anybody can check it in an afternoon without repeating the work that found it. The search here reduces an integer lattice with the Lenstra–Lenstra–Lovász algorithm [4], in an integer-preserving form with the standard three-quarters condition.

Its control is the BBP formula for π [2]. Pointed at base sixteen, power one, stride eight, the pipeline returns an exact relation, and the known formula holds on the same values to 895 bits of 900, the remainder being the series truncation. The gate runs in both directions: it recovers a planted relation exactly and it rejects unrelated numbers.

The relation found is not unique. Bailey's compendium [1] lists the zero relations, BBP-type sums that evaluate to zero, and notes that a relation search returns one unless they are excluded. The exact relations form a lattice and the known formula is one member of it. A control that demands a particular member is the wrong control, and it reports a correct answer as a failure.

2.2 THE MEASUREMENT

One cell, π at base sixteen, power one, stride eight, on the host:

places	seconds	multiplies	mean width	Gram-Schmidt rebuilds
550	1.28	1,048,433	387	1,477
1,100	4.66	1,853,850	730	2,612
2,200	23.32	3,433,155	1,423	4,838
4,400	145.57	6,671,400	2,825	—

Read as growth per doubling of the precision:

doubling	time	exponent	multiplies	mean width
550 to 1,100	3.64×	1.86	1.77×	1.89×
1,100 to 2,200	5.00×	2.32	1.85×	1.95×
2,200 to 4,400	6.24×	2.64	1.94×	1.99×

The exponent is not flat and should not be quoted as though it were. It reads 1.86, then 2.32, then 2.64, rising across the range and still rising at the last step. The figure 2.64 is the largest doubling measured and it is the right one to carry forward, because the smaller cells sit where fixed costs still show. Quoting it as a constant across the whole range would overstate what the four rows support.

2.3 WHICH PART CARRIES THE COST

The natural suspect is the lattice reduction, and the measurement rules it out. Gram-Schmidt rebuilds grow 1.77× then 1.85× per doubling, close to linear in the precision and the same rate the multiply count grows at over the same two steps. The reduction is not outpacing anything.

The multiply carries it. The mean operand width grows 1.89, 1.95, 1.99 per doubling, linear in the precision to within the resolution of four rows. Karatsuba costs $\text{width}^{1.585}$, and a linear width against a roughly linear count of multiplies predicts an exponent of $1 + 1.585 = 2.585$. The measurement returns 2.64.

Agreement to that tolerance on a four-row fit is not a proof of the mechanism, but the mechanism predicts it while the alternative predicts something else. A reduction-bound search would show rebuilds growing faster than the multiply count, and they do not.

2.4 THE CONSEQUENCE

A device multiply beats the host only above operands of 32,768 bits, 1,024 limbs. The crossover depends on how the device is driven: with a fresh device context per call, 192 ms of setup puts it at 16,384 limbs, and one persistent process brings it down to 1,024.

The mean operand in this search is 0.642 times the precision, read directly off the last row, 2825/4400. The precision at which the mean operand reaches the device crossover is therefore

$$32768/0.642 \approx 51{,}000 \text{ places.}$$

Scaling the measured 145.57 s at 4,400 places by $(51000/4400)^{2.64}$ gives about 26 hours for one cell there, and a 240-cell grid is about 261 days.

The precision at which the device becomes the right machine is a precision at which a single cell takes a day. No amount of device is the answer to this search, and that is measured instead of argued: the two thresholds are separated by a factor the hardware does not close, and closing it would need a different multiply instead of a faster one.

2.5 WHAT THIS DOES NOT SETTLE

The exponent is fitted on four rows spanning three doublings, and the largest is the only one near the asymptote. A fifth row would tighten it and is not run.

The crossover at 32,768 bits is a property of one device and one host, measured on one desktop. It moves with either.

And the mechanism argument is an inference from two rates agreeing with a prediction instead of a direct attribution. The reduction is ruled out, since its growth is close to linear and does not outpace the multiply count. What the multiply's exponent is exactly, as against what Karatsuba predicts for it, would need the multiply timed on its own across the same widths.

3

THE RECORD OF CONFIGURATIONS

This chapter states how each configuration of the arbitrary-precision arithmetic is recorded: what is claimed of it, what would refute the claim, and what settles it. It also states the constraints every entry is bounded by. The entries are measured and repeatable; they are not formal theory, and several record the implementation as the thing measured.

3.1 THE SCHEMA

Every entry carries five fields.

field	what it holds
Category	which class the entry belongs to
Hypothesis	one sentence, stated so it can be wrong
Configuration	every parameter needed to re-run it, none implied
Refutation	the result that would kill it, written before the verdict
Proof	what is run, what comes back, which way it settles

Refutation is written before the proof. A hypothesis with no refutation absorbs any result, because nothing is excluded in advance. Only a claim stated sharply enough to be caught wrong can find something.

Configuration is complete or the entry is worthless. A parameter left implied, the machine an entry ran on above all, makes the entry unrepeatable and its result unattributable.

Identifiers are `E-`*`category`*`-`*`number`*, assigned once. A refuted entry retains its number and gains a verdict. Nothing is deleted, since a record of what fails prevents it being retried.

3.2 LOCATED CONSTRAINTS

These bound every entry and are not re-argued in each.

The search is precision bound instead of time bound. A relation whose coefficients run to D digits cannot be seen below roughly $D \cdot n$ digits of input, because the lattice cannot separate a true relation from an accidental cancellation there. Depth is the only axis with no floor under it.

A device multiply beats the host only above 1,024 limbs, which is 32,768 bits. Below that a device call is mostly transfer. Measured on one host and one device, and it moves with either.

One region is closed. Borwein, Galway and Borwein [3] prove that π has no Machin-type BBP arctangent formula in any base $b \neq 2$ that is not a proper power. Searching there returns nothing new, and it appears below only as a control.

A candidate is not a discovery. Surviving the screen shows only that a relation is not a precision artifact. The majority of the searched space is already mapped, and a surviving relation has still to be checked against the compendium [1].

BIBLIOGRAPHY

[1] D. H. Bailey. *A Compendium of BBP-Type Formulas for Mathematical Constants*. 2023.

[2] D. H. Bailey, P. B. Borwein, S. Plouffe. *On the Rapid Computation of Various Polylogarithmic Constants*. Mathematics of Computation 66(218), 903-913, doi:10.1090/s0025-5718-97-00856-9. 1997.

[3] J. Borwein, W. Galway, D. Borwein. *Finding and Excluding b-ary Machin-Type Individual Digit Formulae*. Canadian Journal of Mathematics 56(5), 897-925, doi:10.4153/cjm-2004-041-2. 2004.

[4] A. K. Lenstra, H. W. Lenstra, L. Lovász. *Factoring polynomials with rational coefficients*. Mathematische Annalen 261(4), 515-534. 1982.

[5] F. Proth. *Théorèmes sur les nombres premiers*. Comptes Rendus de l'Académie des Sciences, Paris, 87, 926. 1878.

[6] D. Quigg. *The Twiddle Constants, Proved and Not Copied*. orior research papers, https://github.com/dstroy0/orior. 2026.

[7] Prasanna Ravi, Bolin Yang, Shivam Bhasin, Fan Zhang, Anupam Chattopadhyay. *Fiddling the Twiddle Constants: Fault Injection Analysis of the Number Theoretic Transform*. IACR Transactions on Cryptographic Hardware and Embedded Systems 2023(2), 447-481; IACR ePrint 2022/824. 2023.